

Architecture Write-Up

1) System Design:

- The workflow was designed as an end-to-end AI intake and triage pipeline for ArcVault using n8n for orchestration, Groq's model for language tasks, and Google Sheets for persistent structured output storage. The main workflow begins with a manual trigger and a sample-message loader so the five required synthetic inputs can be processed consistently during testing and demonstration. Those messages are then split into individual items so each request can move through the same processing path independently. In addition to the main workflow, an optional Gmail-triggered version was built to show how the same pipeline could start from a real incoming email.
- The processing path is structured into clear stages. First, the classification step sends each message to the LLM and returns a constrained JSON object containing category, priority, and confidence score. Second, the enrichment step sends the original message and extracts the core issue, identifiers, and urgency signal in a structured format. Third, a deterministic routing and escalation step is handled in n8n using code-based logic rather than leaving the final routing decision to the LLM. Fourth, a summary generation step produces a two-sentence human-readable handoff note for the receiving team. Finally, the output is flattened into a clean row structure and written to Google Sheets, where the final state is stored and can be reviewed easily.
- State is held in two places. During execution, intermediate state is carried through n8n from node to node as JSON. Persistent state is held in Google Sheets, which stores the final structured output for all requests. This design keeps the workflow simple while still making the final results visible and shareable.

2) Routing Logic

- The routing logic was intentionally implemented outside the LLM to make it more transparent, predictable, and easier to explain. The LLM is used to classify the request, but the actual destination queue is determined using explicit business rules in n8n. This reduces the chance of inconsistent downstream decisions and makes the workflow easier to debug.

- The queue mapping is as follows. Bug Reports and Incident/Outage requests are routed to Engineering because both relate to product behavior, breakages, or service availability. Feature Requests are routed to Product because they represent new capability requests rather than defects. Billing Issues are routed to Billing because they involve charges, invoices, or contract-rate mismatches. Technical Questions are routed to IT/Security because the sample scenario included configuration and authentication topics such as SSO and Okta. A fallback queue of Human Review was included as a safe default in case classification was uncertain or a future category was ambiguous.
- This mapping was chosen because it mirrors how many SaaS companies separate operational responsibilities. It also keeps the workflow easy to understand during review: the LLM decides what the request is, and the routing logic decides who should act on it.

3) **Escalation Logic:**

- The escalation logic was designed to identify cases where a normal queue assignment might not be sufficient. Two main escalation paths were implemented. The first is model uncertainty: if the confidence score is below 70, the item is flagged for human review. This prevents low-confidence classifications from being handled automatically without oversight. The second is content-based escalation for operationally sensitive cases.
- The workflow checks for incident-like language such as “outage,” “down for all users,” “multiple users affected,” or similar wording that suggests broad customer impact. It also calculates billing discrepancies when two monetary values appear in a billing message, and escalates the case if the difference is greater than \$500. These rules were selected because they match the examples given in the assessment and because they represent situations where speed or caution matters more than automation convenience.
- When escalation is triggered, the standard route is overridden and the item is sent to a separate Escalation Queue. This makes the workflow safer and demonstrates that routing is not purely linear; exceptional cases are treated differently.

What Would Be Different at Production Scale:

- At production scale, the workflow would need stronger reliability, lower latency, and better operational safeguards. First, secrets would be managed through credentials or environment variables only, never in workflow nodes. Second, schema validation would be added after every LLM call so malformed JSON could be retried or quarantined automatically. Third, retries, timeout handling, and dead-letter storage would be added for failed API calls or downstream write failures.
- Cost and latency would also need closer management. The current design uses multiple LLM calls per request because that made the logic easier to separate and explain. In production, some of these steps could be combined or selectively skipped depending on message type to reduce token usage and response time. Monitoring would also be necessary, including success rates, average run time, escalation volume, and model error patterns. Finally, Google Sheets is fine for assessment visibility, but a production version would likely write to a database or ticketing system such as Zendesk, Jira, or a queue-backed service.

Phase 2 Improvements

- With another week, the next phase would focus on making the workflow more realistic and more resilient. The first addition would be support for more ingestion paths, such as webhooks, forms, and support-platform events, all normalized into the same internal request schema. The second would be a dedicated escalation tab or store with alerts so high-risk cases are easier to track. The third would be better confidence calibration and example-based prompting to reduce overconfident scores.
- A Phase 2 version would also add downstream actions. For example, it could automatically create tickets in the correct system, attach the structured summary, and notify the right team. Another useful addition would be a lightweight knowledge lookup step for technical questions so the workflow could enrich responses with known configuration guidance before routing. Overall, the next phase would focus on moving from a working prototype to a more robust operational workflow.